

Benchmarking and Performance Analysis of LLM Inferencing Systems

Saksham Rathi

Systems for LLMs
Under Prof. Mythili Vutukuru

Indian Institute of Technology Bombay
Autumn 2025-26

Contents



1. Various Scheduler Designs
   - vLLM Schedulers
   - Our Schedulers
   - Already Existing Schedulers
   - SGLANG Schedulers
2. Throughput Numbers
3. Key Propositions
4. L2Cache Simulator

First Come First Serve Policy



- Adds requests one by one, from the core engine into a waiting queue
- The requests are extracted in a first come first serve manner
- We stop selecting requests from the waiting queue into the running batch, once the size reaches `max_batch_size`

Priority Based

- The user passes priority values along with the requests' strings themselves
- We use a min heap to find the request with the smallest priority value (executed at first)
- Ties are broken in a first come first serve manner (through tracking request's arrival time)

Greedy First Come First Serve



- We keep a track of the working set along with the requests we add to the current running scheduler batch
- We add requests in a first come first serve manner
- However, now we are also controlling the batch size - we stop whenever addition of the next request increases the working set size beyond a threshold (decided by L2 cache size)
- For our workload, and GPU - it boils down to stopping whenever the next request belongs to a different family

Greedy First Come First Serve - Appendix - Expected Batch Size



Let's say we have N families. The user passes a bunch of requests, which can belong to any of these families. Moreover, all the requests have been shuffled, such that the probability of a request at position i to be of family j is exactly $\frac{1}{N}$.
Assume that the first request belongs to family j. We find out the expected value of the size of the first scheduler batch.
For the batch to have size exactly x:

- The first $x-1$ requests after the initial one must also be of family j.
- The x-th request must be of a different family.

The probability that the batch is of size x is given by:

$$\Pr[X = x] = \left(\frac{1}{N}\right)^{x-1} \cdot \frac{N-1}{N}, \quad x \geq 1.$$

Greedy First Come First Serve - Appendix - Expected Batch Size



Let X denote the size of the first batch. Its expected value is:

$$\mathbb{E}[X] = \sum_{x=1}^{\infty} x \Pr[X = x] = \sum_{x=1}^{\infty} x \left(\frac{1}{N}\right)^{x-1} \frac{N-1}{N}.$$

Using the identity

$$\sum_{x=1}^{\infty} x r^{x-1} = \frac{1}{(1-r)^2}, \quad |r| < 1,$$

with $r = \frac{1}{N}$, we obtain:

$$\mathbb{E}[X] = \frac{N-1}{N} \cdot \frac{1}{\left(1 - \frac{1}{N}\right)^2} = \frac{N}{N-1}.$$

$$\boxed{\mathbb{E}[\text{batch size}] = \frac{N}{N-1}}$$

For example, when we have 2 families, the expected batch size would be just 2.

Family ID Based



- If we can set priority values for each of the request, then why can't we set the family to which they belong to?
- We have separate queues for each family
- All the individual queues operate in a first come first serve manner
- If there is a request of family ID j already present in the scheduler batch, then we try to add all the requests of that family in the same batch (till we hit `max_batch_size`). If there is no request of that family available, then we stick to the current batch (no more requests added)
- So, the batch size is no longer static - it is dynamically set in accordance with the workload

Radix Tree Based



- Key idea: Instead of treating each request independently, we exploit shared token prefixes across requests
- Each request is represented as a path of tokens in a radix tree
- Internal nodes correspond to shared token substrings
- The scheduler maintains a working set of active nodes (tokens already materialized / cached)
- Each waiting request has a cost = number of tokens missing from the working set

Radix Tree Based



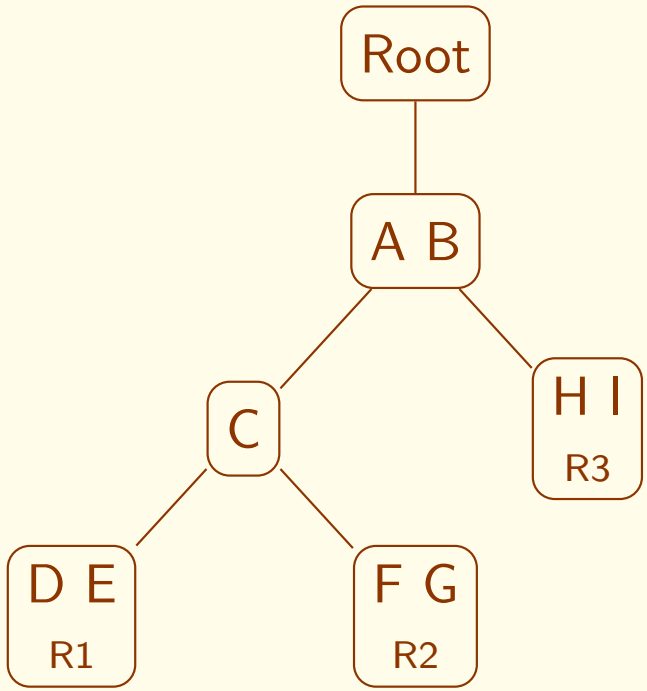
To schedule the next request:

- We start from leaves already in the working set
- Traverse upwards toward the root
- At each node, we look for waiting requests passing through it
- We select the request with minimum missing-token cost

Activating a request:

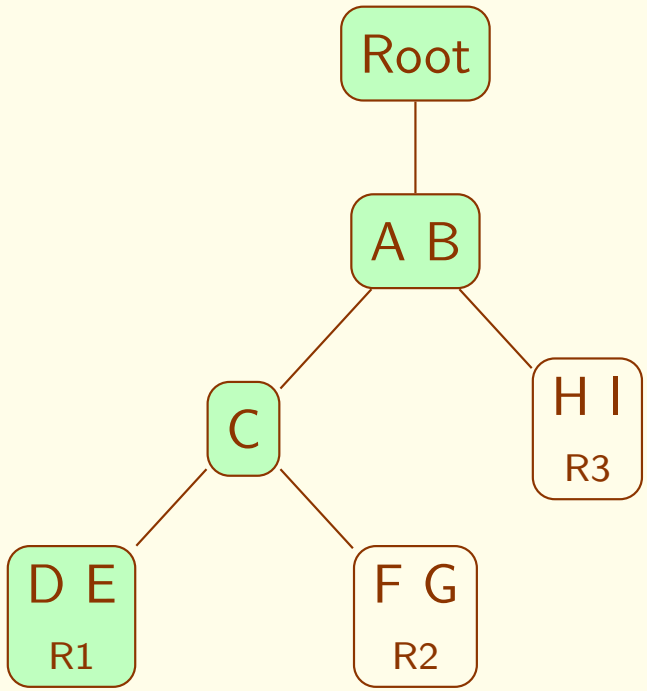
- Adds its path to the working set
- Future requests that share prefixes become cheaper
- This dynamically forms prefix-coherent batches, minimizing recomputation

Radix Tree Based - Radix Tree Construction



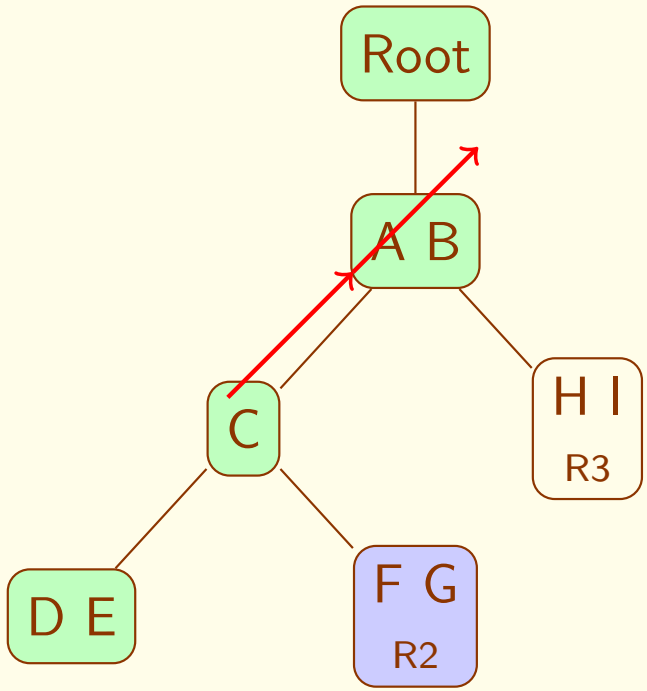
Each leaf corresponds to a request. Internal nodes represent shared token prefixes.

Radix Tree Based - Working Set After Activation



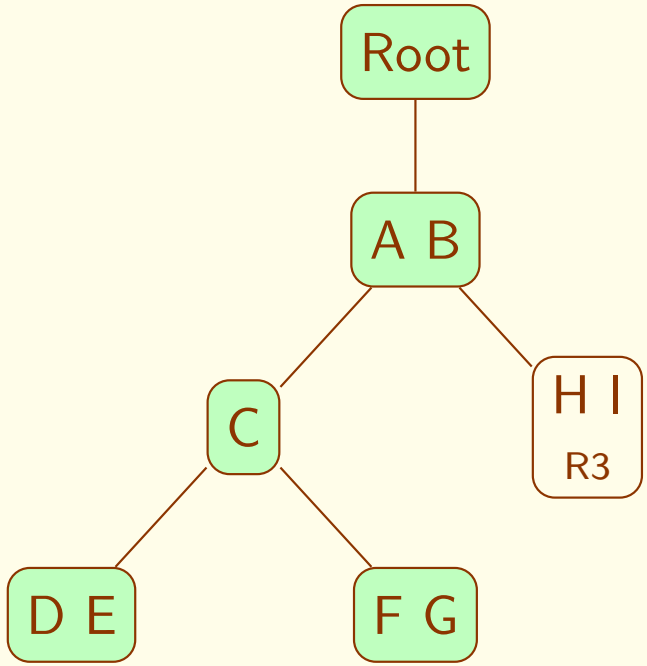
Green nodes indicate the working set after activating R1.

Radix Tree Based - Finding the Best Request



Traversal starts from working-set leaves and moves upward. R2 is selected due to maximum prefix overlap.

Radix Tree Based - Dynamic Batch Growth



Batch size grows dynamically as long as marginal cost is low.

Radix Tree Based - Issues



The above approach has a few issues:

- Adding a request is quite costly - there might be splitting/merging of nodes. Although finding the best request is quite efficient
- Each request may contain thousands of tokens, so the tree depth might get quite large too
- We cannot have a parallel thread/process handling the radix tree, because maintaining sync is difficult. What if we removed a request, from our waiting queue, but it takes some time to remove it from the radix tree?

Chunked Hash Tree Based



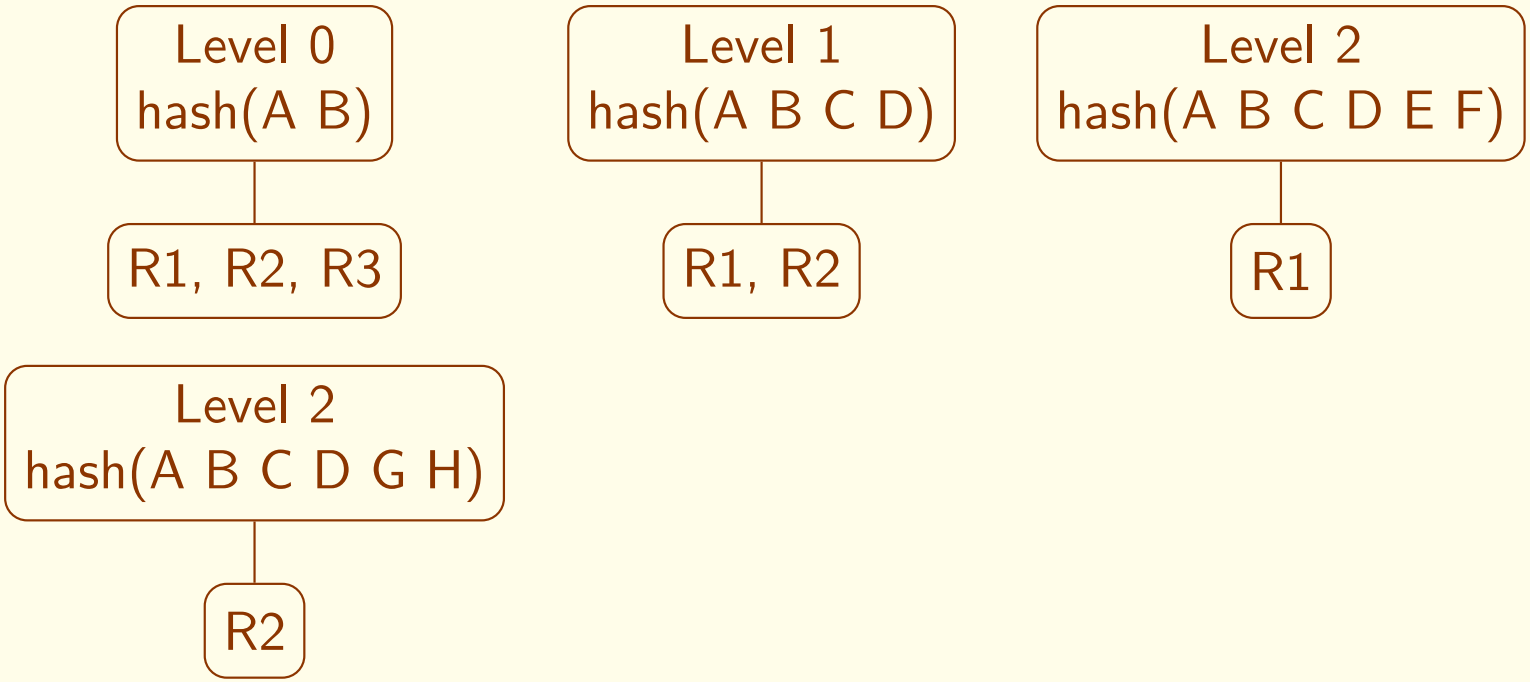
- Requests are represented using **hashed cumulative prefixes**
- Tokens are divided into fixed-size chunks
- Each level corresponds to `hash(tokens[0:k])`
- No explicit tree structure or node splitting
- Much faster than radix trees for large workloads

Chunked Hash Tree Based - Working Set



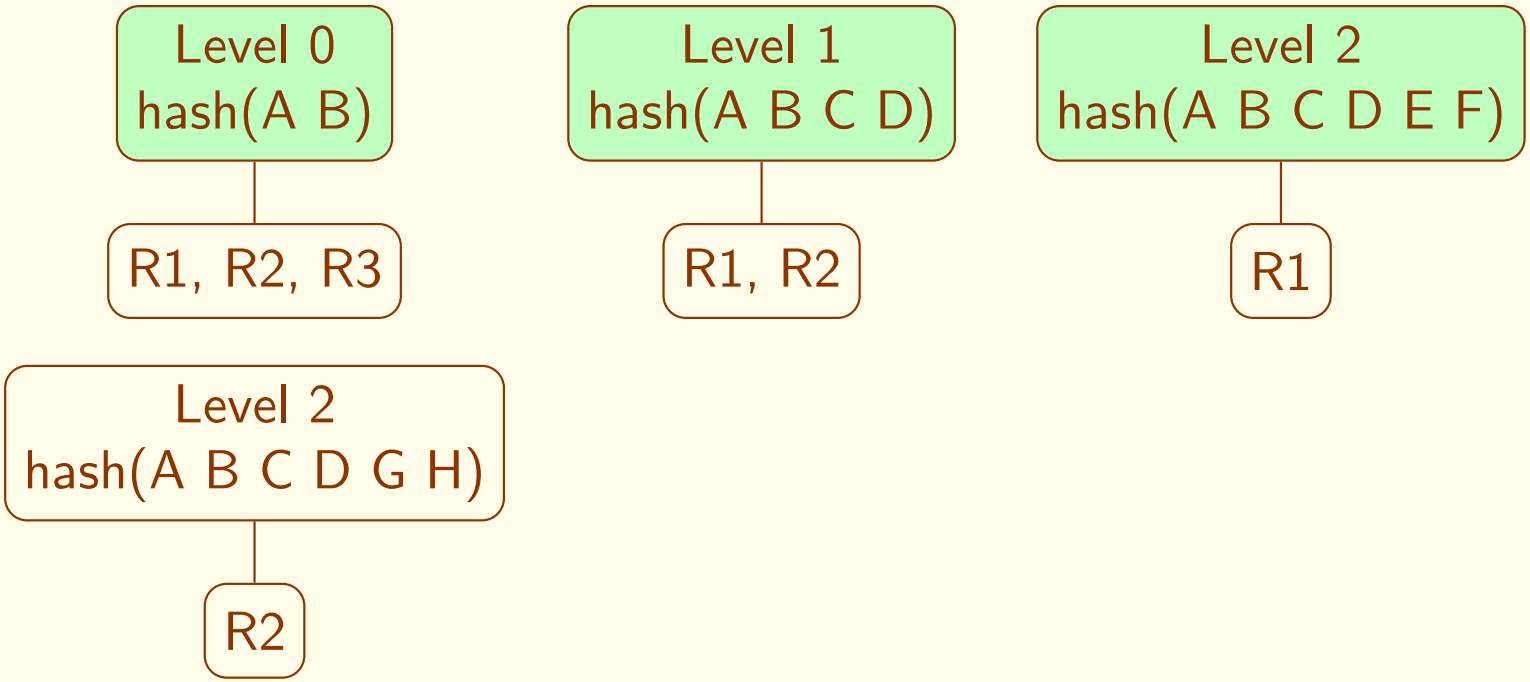
- Working set = set of active **(level, hash)** pairs
- Each active request contributes all its prefix hashes
- Request cost = number of missing prefix hashes
- Scheduling prefers requests with minimal missing chunks
- Enables dynamic batch growth similar to radix trees

Chunked Hash Tree Based - Insertion



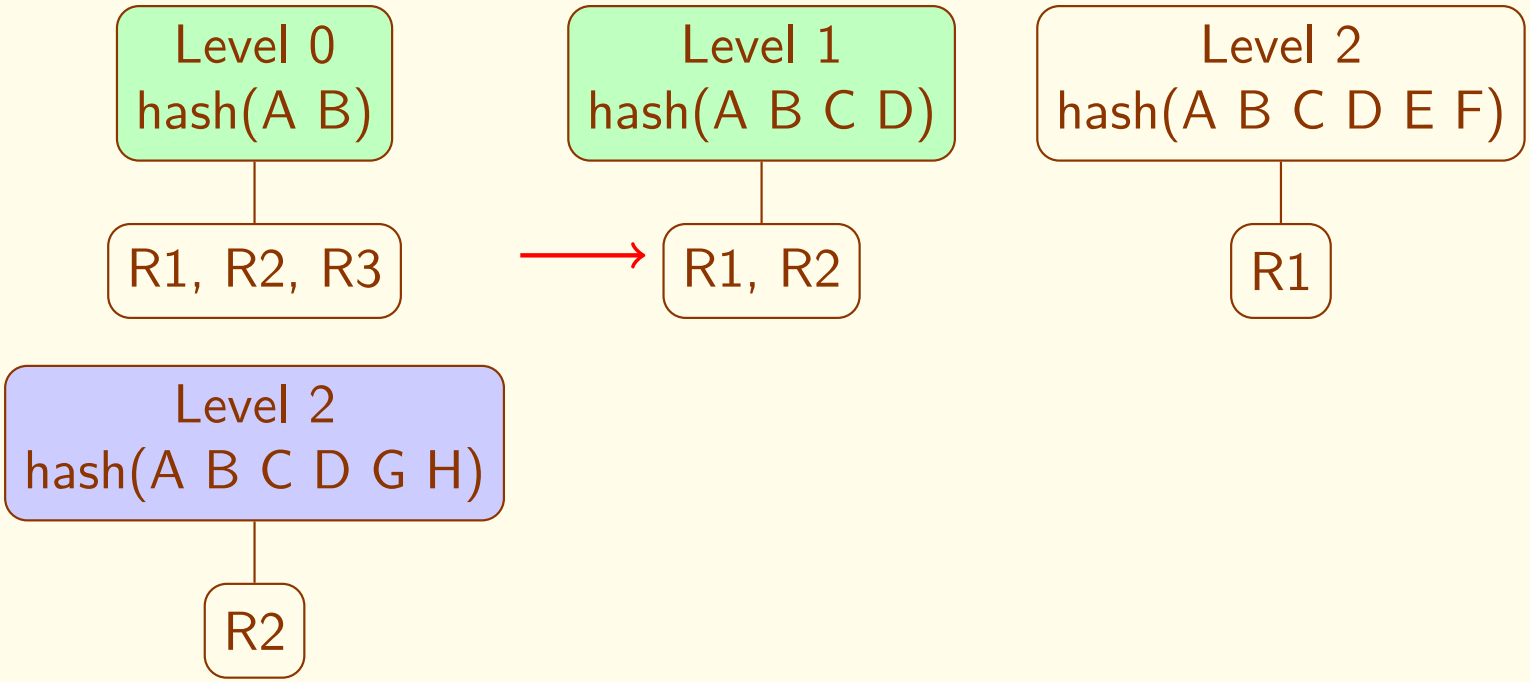
Each level stores hashes of cumulative token prefixes.

Chunked Hash Tree Based - Working Set



Green boxes indicate prefix hashes in the working set.

Chunked Hash Tree Based - Best Request Selection



R2 is selected since it has maximum prefix overlap with the working set.

Dynamic Batching



- Static batching uses a fixed maximum batch size
- Dynamic batching adapts batch size based on **runtime GPU memory state**
- Memory usage is dominated by KV cache allocation
- Larger batches improve throughput but increase memory pressure

Dynamic Batching - Memory-Aware Control



- Let B be the current batch size
- Measure used GPU blocks and estimate blocks per sequence:

$$\text{blocks_per_seq} = \frac{\text{used GPU blocks}}{B}$$

- Compute the memory-safe maximum batch size:

$$B_{\max} = \left\lfloor \frac{\text{total GPU blocks} \times \alpha}{\text{blocks_per_seq}} \right\rfloor$$

- α is a safety factor (e.g., $\alpha = 0.95$)
- Stop adding requests when $B \geq B_{\max}$

BucketServe - Bucket-Based Dynamic Batching



- Real-world workloads exhibit **highly heterogeneous sequence lengths**
- Naive batching mixes short and long requests, causing:
  - Excessive padding
  - GPU memory waste
  - Lower effective throughput
- Key idea: **group requests by sequence length** before batching
- Enables tighter memory bounds and more predictable batch sizing

BucketServe: Design



- Incoming requests are assigned to **length-homogeneous buckets**
- Each bucket covers a sequence length interval:

$$[L_b, U_b)$$

- Buckets reduce padding by minimizing (S is the sequence length):

$$S_{\max} - S_{\text{avg}}$$

- Dynamic batching is applied **within each bucket**
- Batch size is adjusted based on:
  - GPU memory availability
  - Maximum sequence length in the bucket

BucketServe : Memory Safety



- Bucket boundaries are adjusted online based on workload distribution
- Buckets are:
  - **Split** when length variance is high
  - **Merged** under low load to reduce scheduling overhead
- For each bucket, compute a memory-safe batch size:

$$N_{\max} = \max \left\{ N \;\middle|\; \sum_{i=1}^{N} S_i \leq M_{\text{safe}} \right\}$$

- Not too relevant for our workload, because the query sizes are roughly similar, and system prompt is anyway shared, so the expected throughput is quite similar to that of vLLM baseline

LPM: Longest Prefix Matching



- Prioritizes requests based on how much of their input prefix already exists in the prefix cache
- Requests with longer cached prefixes are scheduled earlier
- If multiple requests share a short prefix that is not yet cached, only a few are scheduled first. Others may be temporarily deprioritized to allow one request to populate the cache
- If the waiting queue is large (e.g., > 128 requests), LPM is disabled and falls back to FCFS to avoid expensive prefix matching

DFS Weight



- Organizes requests according to the structure of the prefix cache tree
- Each request is associated with a last matched node in the prefix tree
- Each tree node is assigned a weight equal to the number of requests ending at that node plus the weights of all its descendant nodes
- Perform DFS: add requests in that order (visit children in descending order of weights)

Workload Setup



- Two families (each of system prompt of length 1300 tokens)
- 2500 queries of each family - total 5000 queries
- All randomly shuffled
- L2 Cache size = 96 MB (RTX 6000 Ada Generation)
- Each family's system prompt's KVCache roughly takes 180 MB (model: Llama3.1-8B)
- We measure end-to-end throughput (including the time taken to add/process requests)
- We generate 50 new tokens for each request

Why just 50 new tokens?



- LLMs hardly answer in 1-2 sentences, so less than 50 (like somewhere around 10-20) does not make much sense
- The scheduler changes help increasing L2 purity, even during decodes, so larger number of output tokens show better throughput improvement
- Having 200 new tokens for a shared prefix of length 1300 tokens reduces the amount of overlap between two queries

Throughput



Scheduler	Throughput (req/sec)	Comments
First Come First Serve	45.49	vLLM Baseline
First Come First Serve - Family Ordered	64.97	User orders the requests by their families ; theoretical maximum which a user can get from vLLM through client side changes
Greedy First Come First Serve	2.04	Due to very small batch size
Family ID Based	86.72	The fastest which we can achieve ; although through demanding additional information from the user
Radix Tree Based	53.06	Inefficient design ; too much CPU overhead ; consumes a lot of time in between scheduler iterations
Chunked Hash Tree Based	77.44	A much more efficient design ; quite less overhead ; no extra information needed from the user
Dynamic Batching	46.98	Max batch size set to 1000

Table: Throughput Across Different Scheduler Designs

Throughput (for max new tokens = 100)



Scheduler	Throughput (req/sec)
First Come First Serve	20.84
First Come First Serve - Family Ordered	33.94
Family ID Based	42.73
Chunked Hash Tree Based	43.20

Table: Throughput Across Different Scheduler Designs

Throughput (for max new tokens = 200)



Scheduler	Throughput (req/sec)
First Come First Serve	10.67
First Come First Serve - Family Ordered	18.78
Family ID Based	21.14
Chunked Hash Tree Based	21.34

Table: Throughput Across Different Scheduler Designs

Throughput vs Sequence length

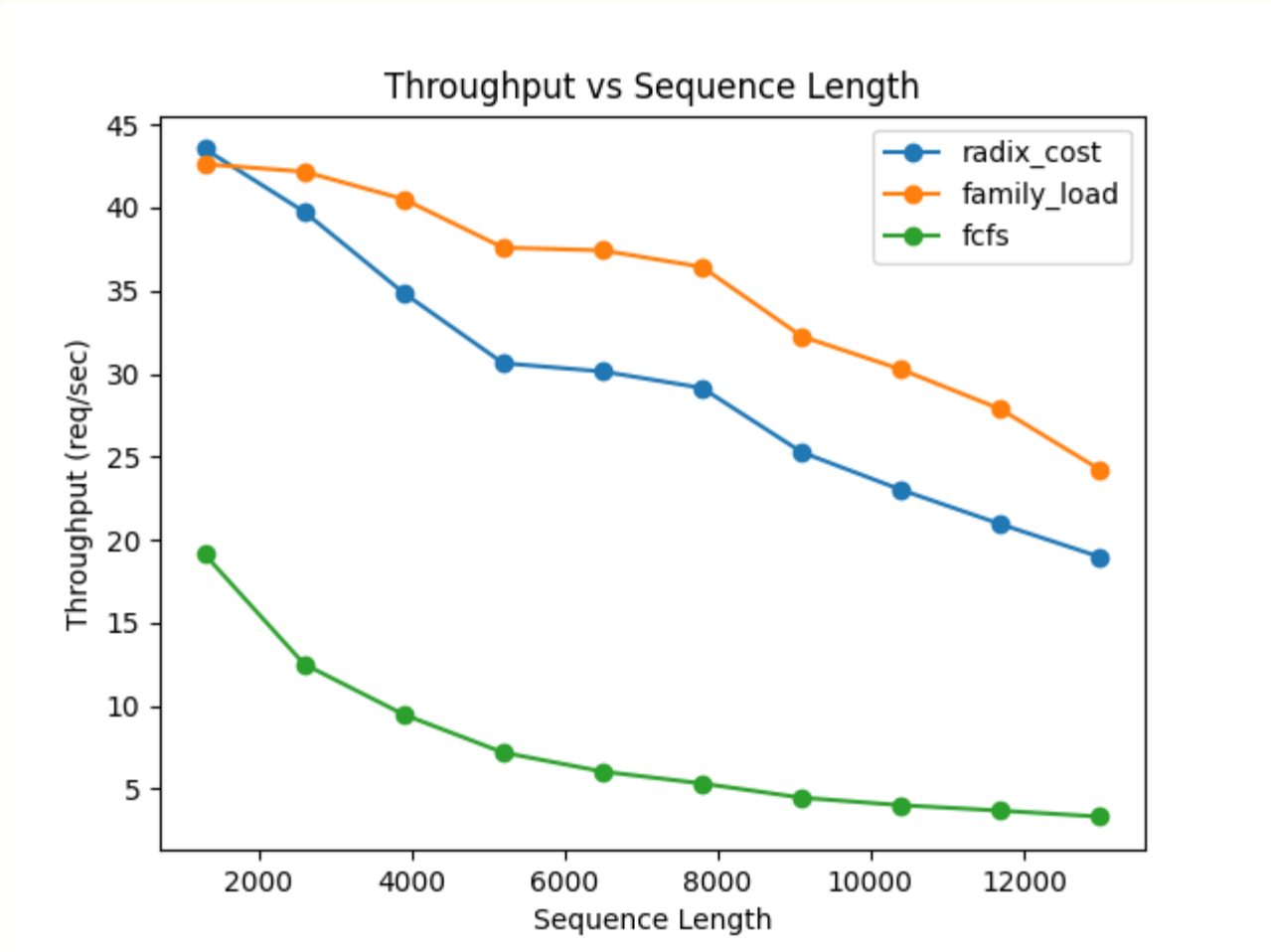


Figure: Throughput vs Sequence Length

Throughput vs req/sec

We model the request ingestion as a poisson process (basically, with exponential sleep times between two consecutive requests)

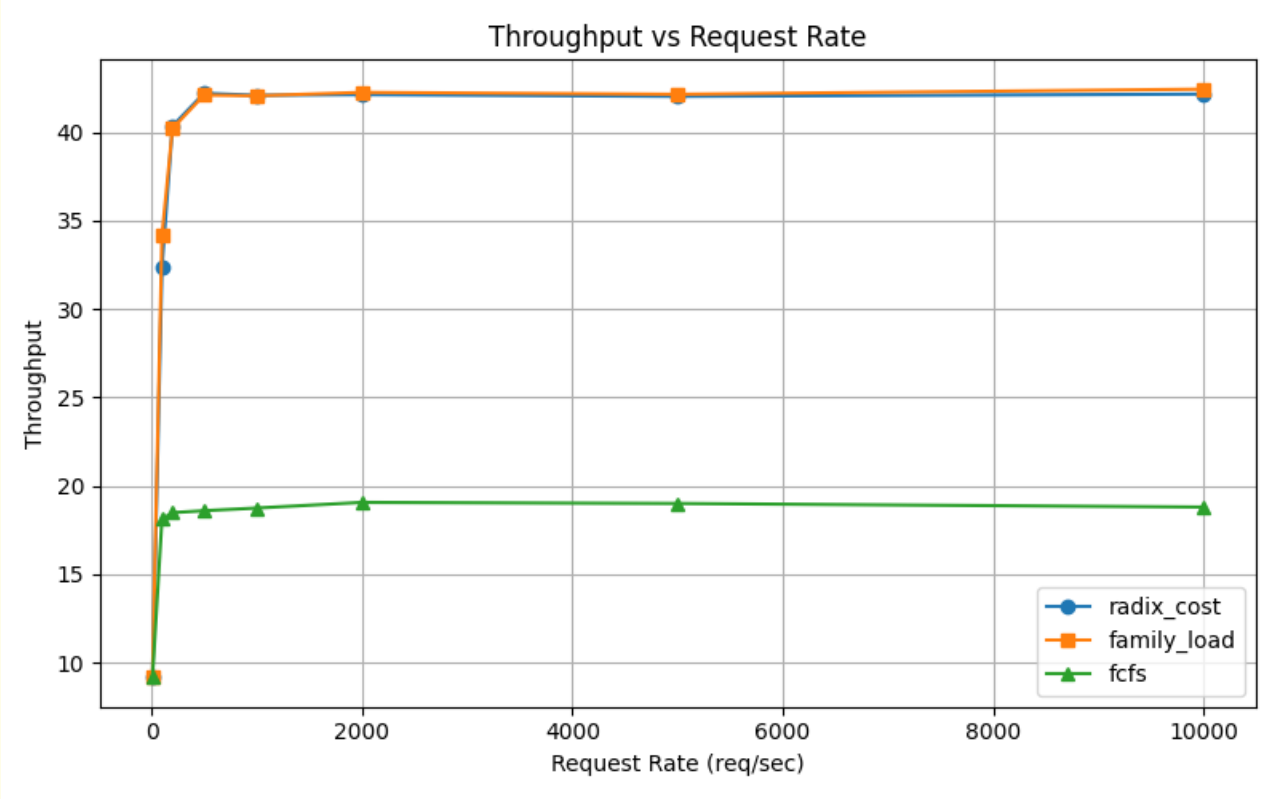


Figure: Throughput vs Varying Ingestion

Throughput vs Sequence length (vLLM vs SGLANG)

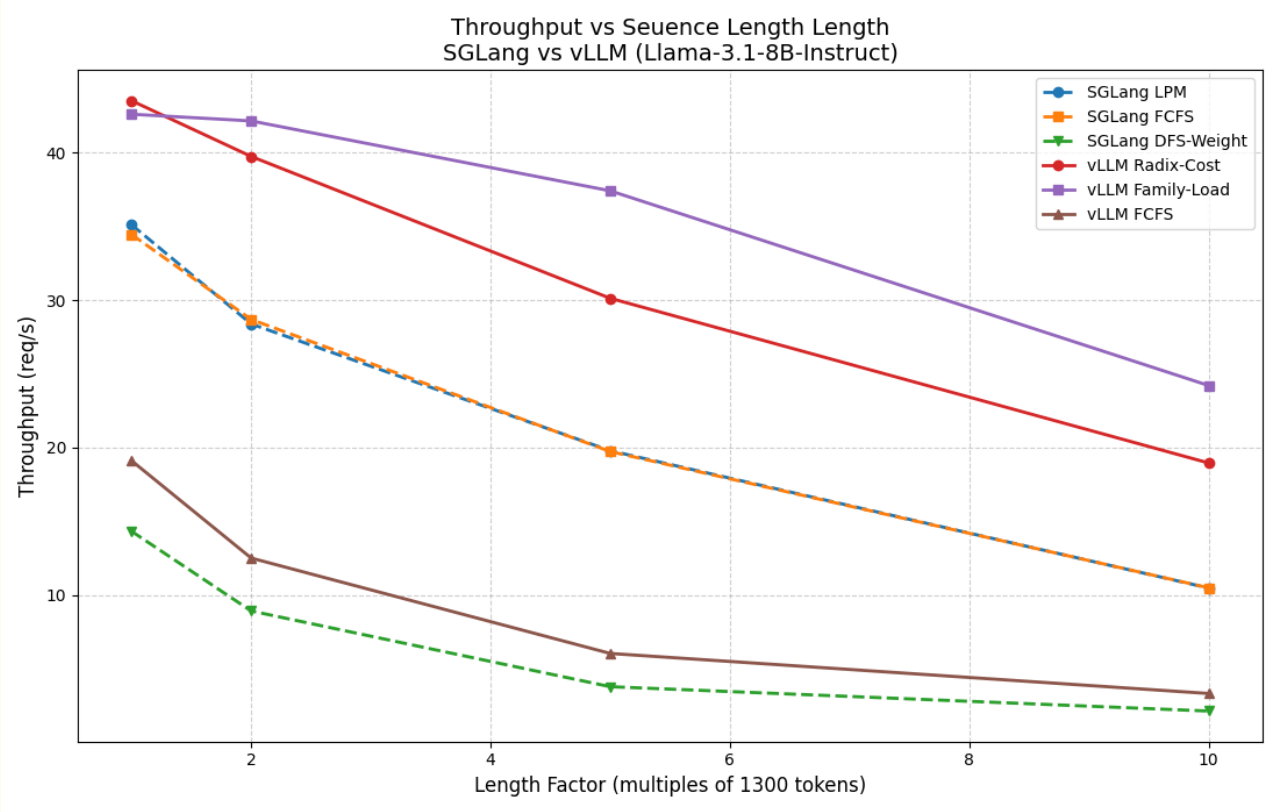


Figure: Throughput vs Sequence Length (vLLM vs SGLANG)

Systems for LLMs
Saksham Rathi
Various Scheduler Designs
Throughput Numbers
Key Propositions
L2Cache Simulator

Throughput vs Req/sec (2 families)

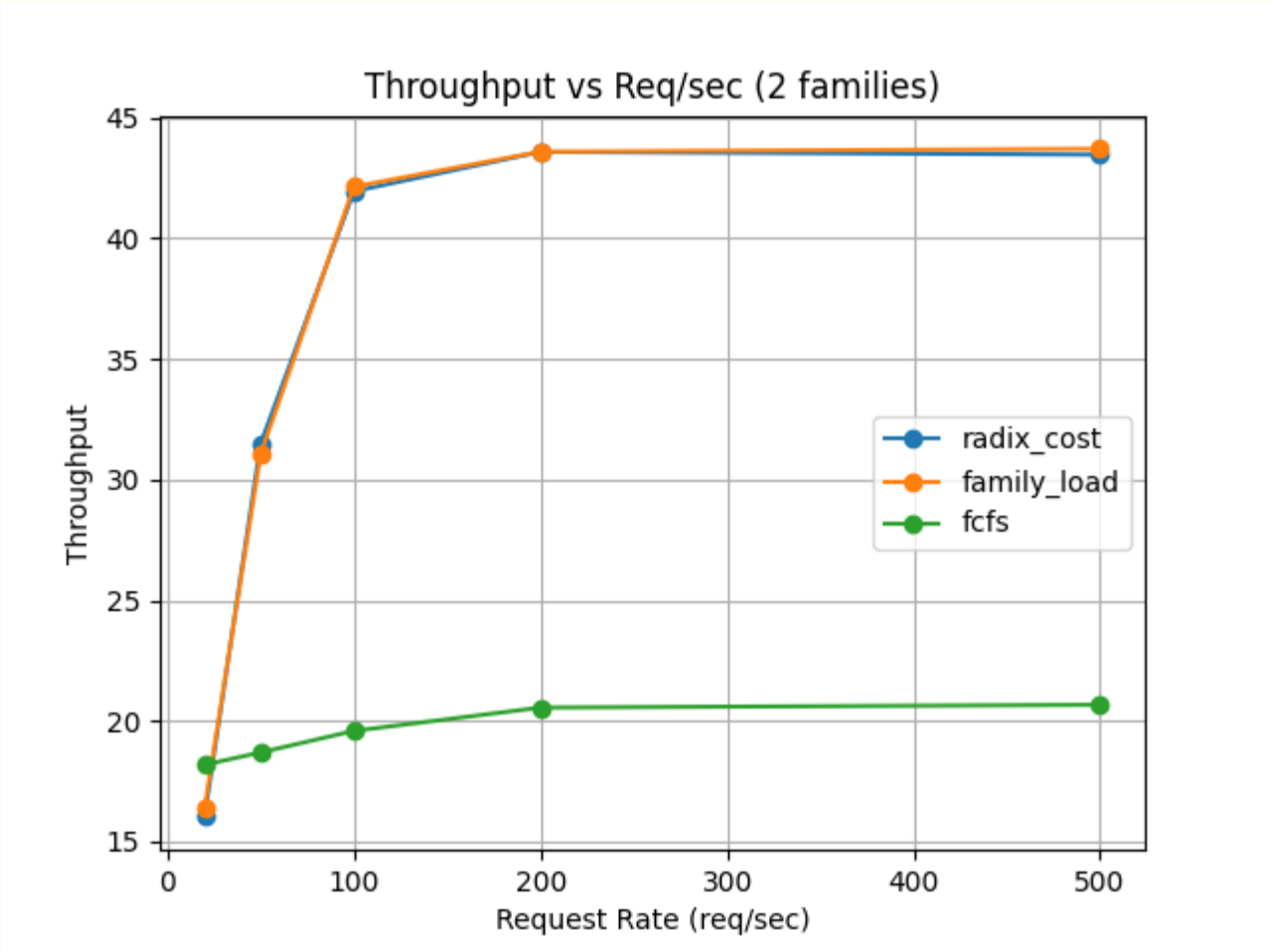


Figure: Throughput vs Req/sec (2 families)

Throughput vs Req/sec (5 families)

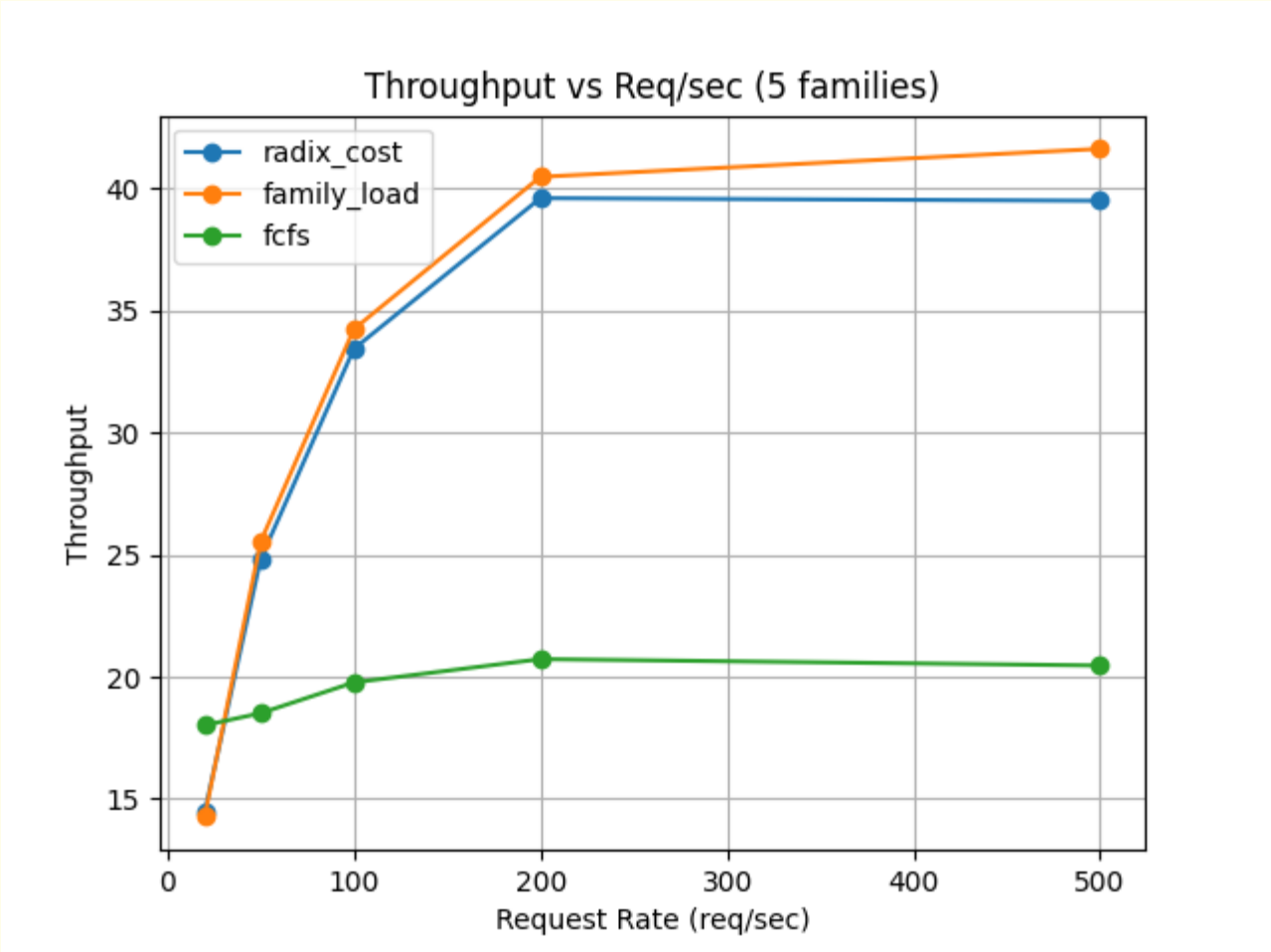


Figure: Throughput vs Req/sec (5 families)

Throughput vs Req/sec (10 families)

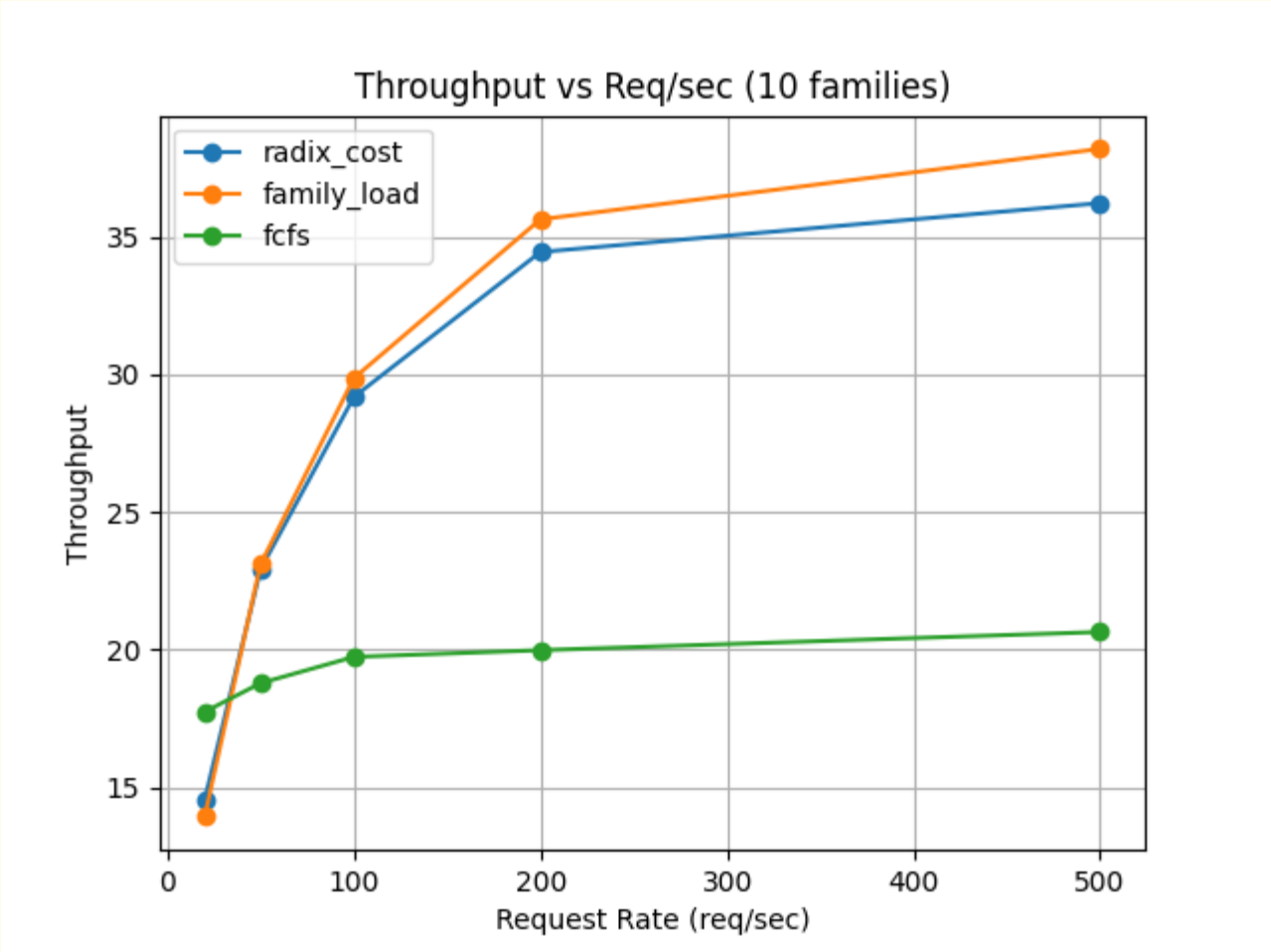


Figure: Throughput vs Req/sec (10 families)

Throughput vs Req/sec (20 families)

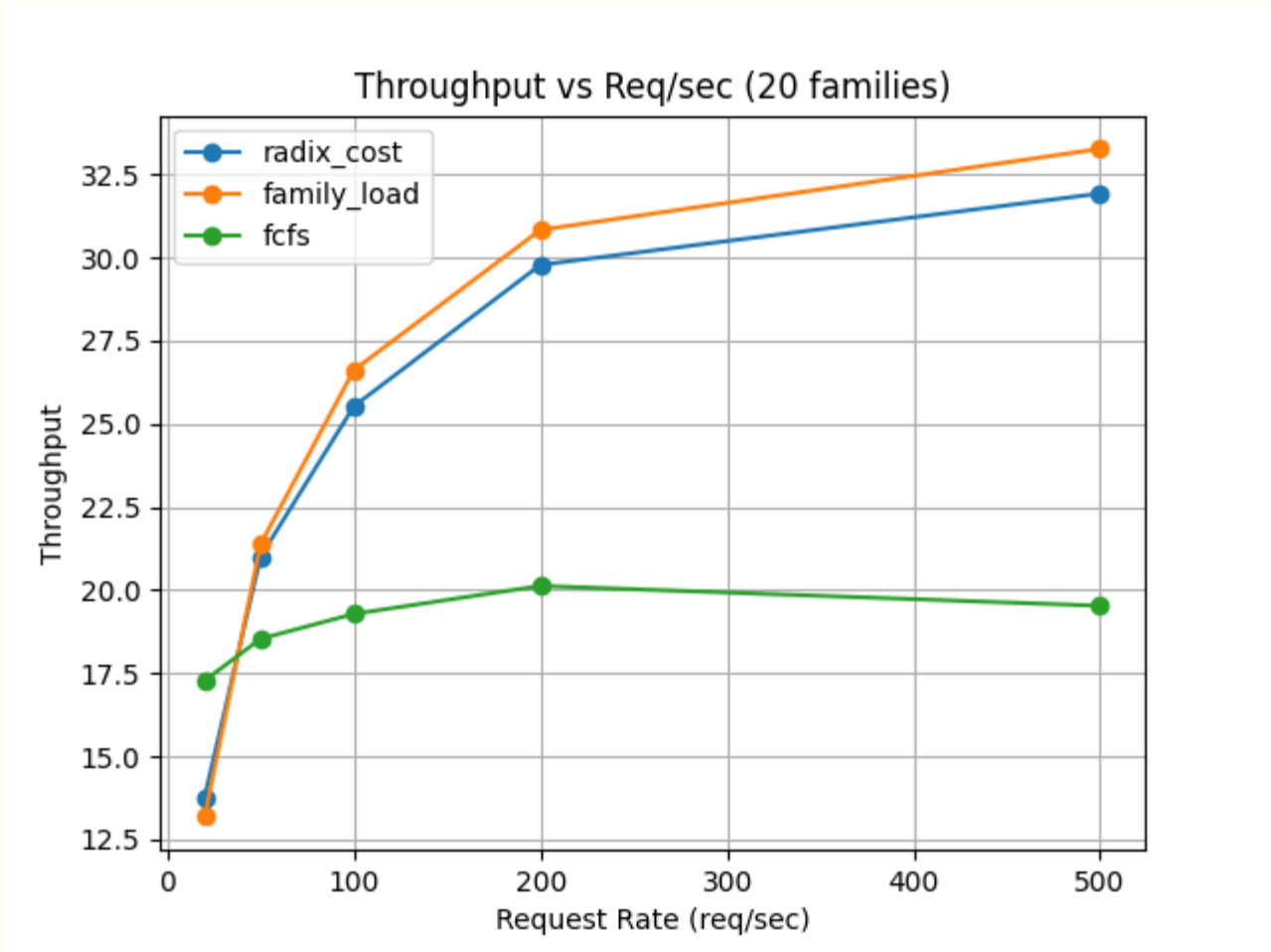


Figure: Throughput vs Req/sec (20 families)

DRAM Metric



Memory BW Utilization	The fraction of cycles where data was sent to or received from device memory. The value represents an average over a time interval and is not an instantaneous value. Higher values indicate higher utilization of device memory. An activity of 1 (100%) is equivalent to a DRAM instruction every cycle over the entire time interval (in practice a peak of ~0.8 (80%) is the maximum achievable). An activity of 0.2 (20%) indicates that 20% of the cycles are reading from or writing to device memory over the time interval.	PROF_DRAM_ACTIVE (ID: 1005)

Figure: DRAM Accesses' Metric (from DCGMI)

DRAM Accesses' CDF

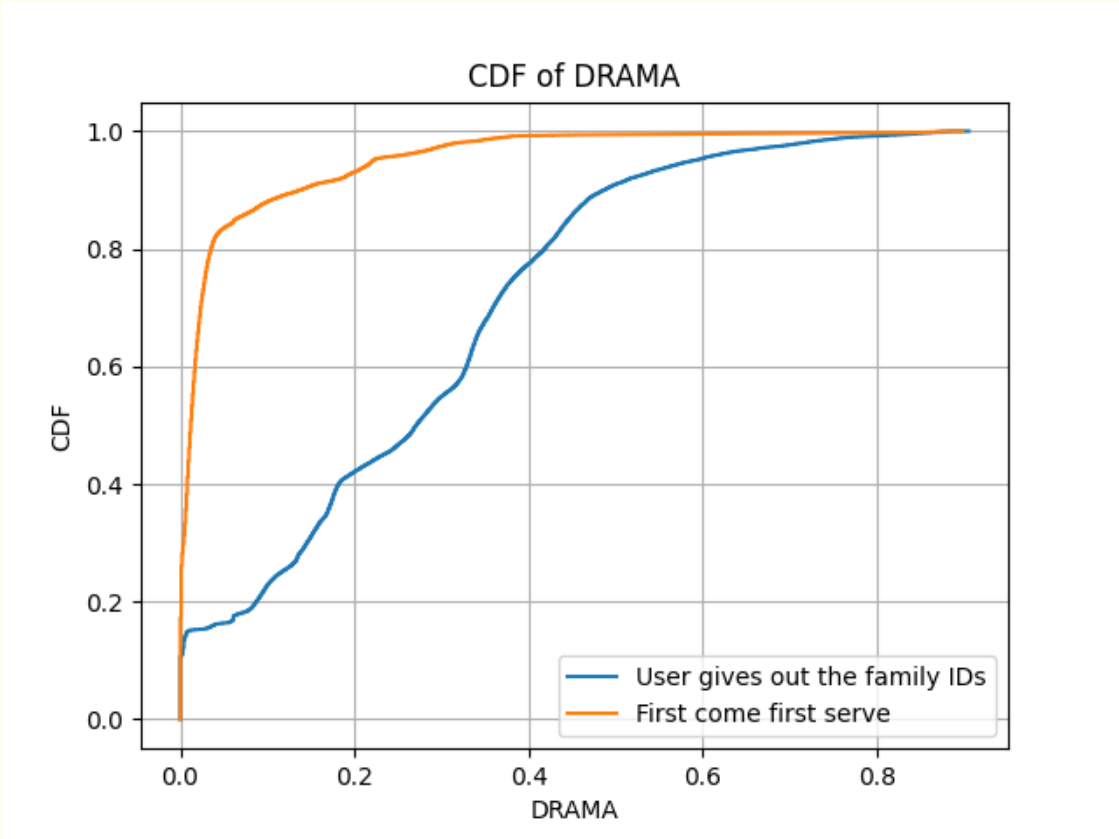
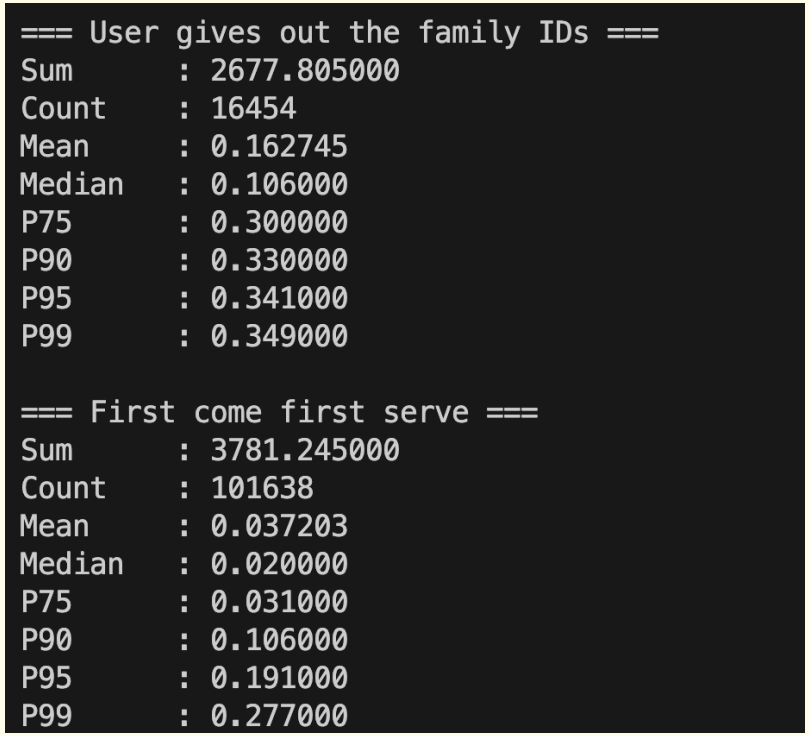


Figure: DRAM Accesses' CDF (2 families, 6500 tokens shared context, throughput gain was roughly 4x, metric sampled every 1ms)

DRAM Accesses' Statistics



```
=== User gives out the family IDs ===
Sum      : 2677.805000
Count    : 16454
Mean     : 0.162745
Median   : 0.106000
P75      : 0.300000
P90      : 0.330000
P95      : 0.341000
P99      : 0.349000

=== First come first serve ===
Sum      : 3781.245000
Count    : 101638
Mean     : 0.037203
Median   : 0.020000
P75      : 0.031000
P90      : 0.106000
P95      : 0.191000
P99      : 0.277000
```

Figure: DRAM Accesses' Stats (2 families, 13000 tokens shared context, metric sampled every 1ms)

Batch Decomposition - 200 requests - single family

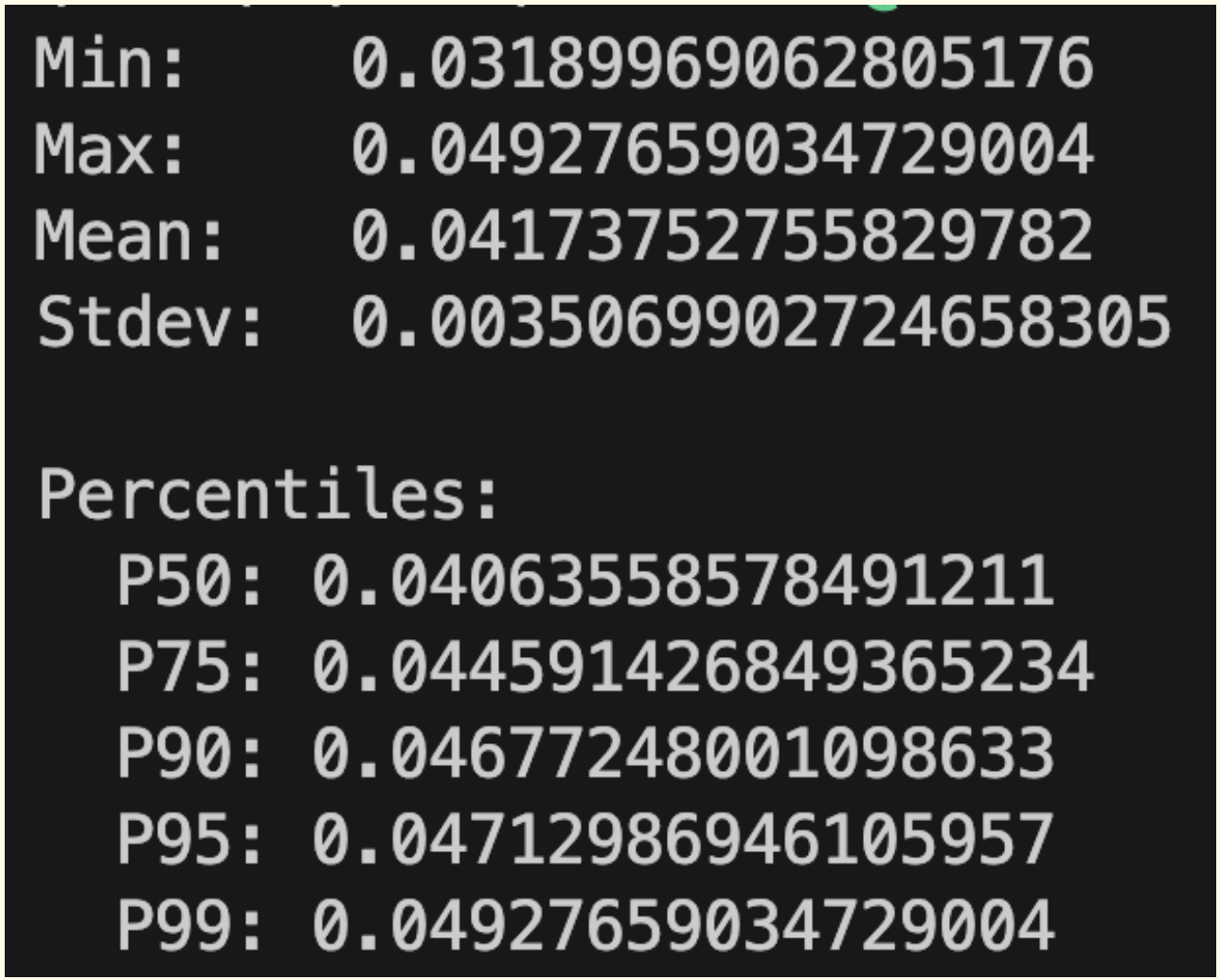


Figure: Decode Stats, 200 requests, single family, 1300 tokens shared context

Batch Decomposition - 200 requests - two families

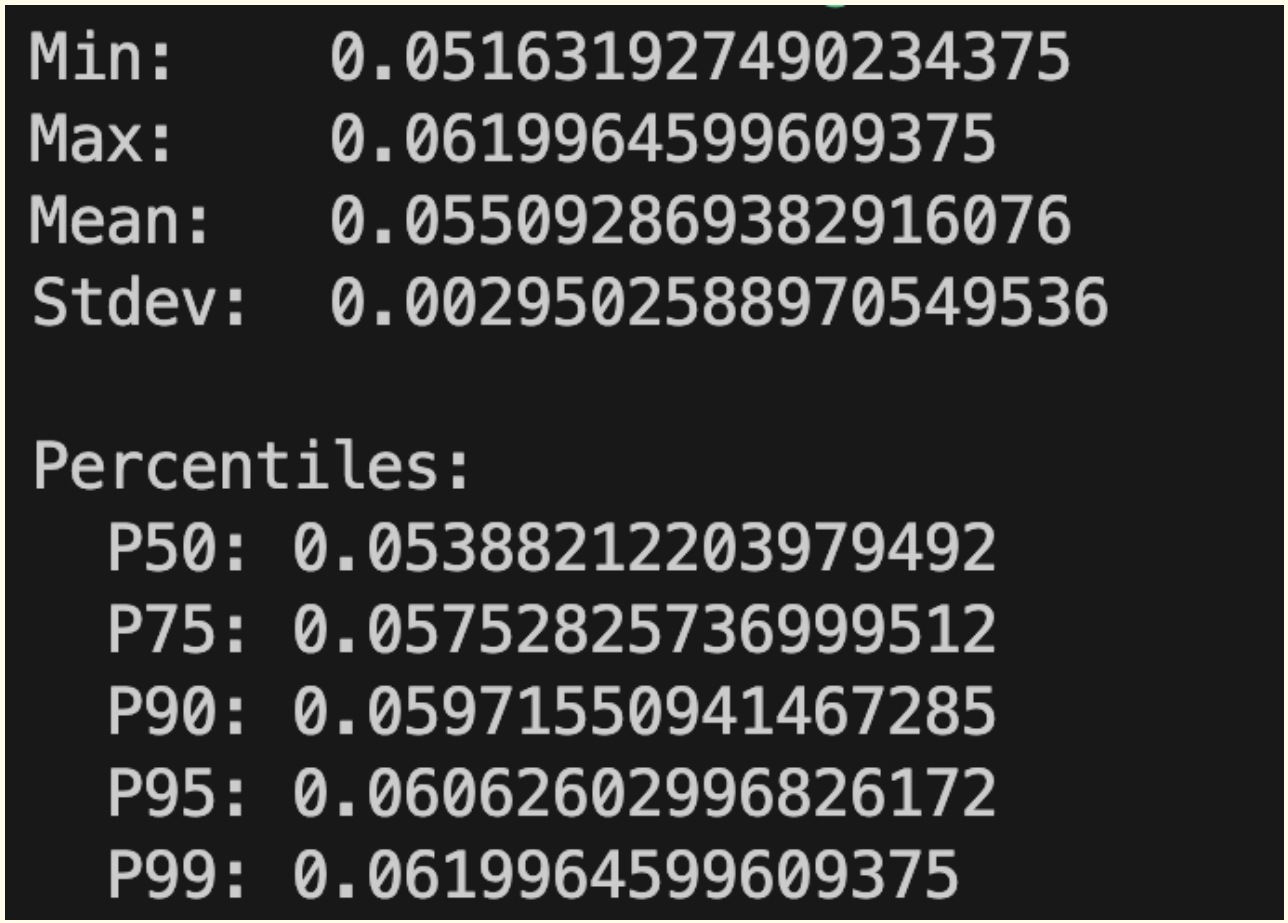


Figure: Decode Stats, 200 requests, two families, 1300 tokens shared context

Batch Decomposition - 500 requests - single family

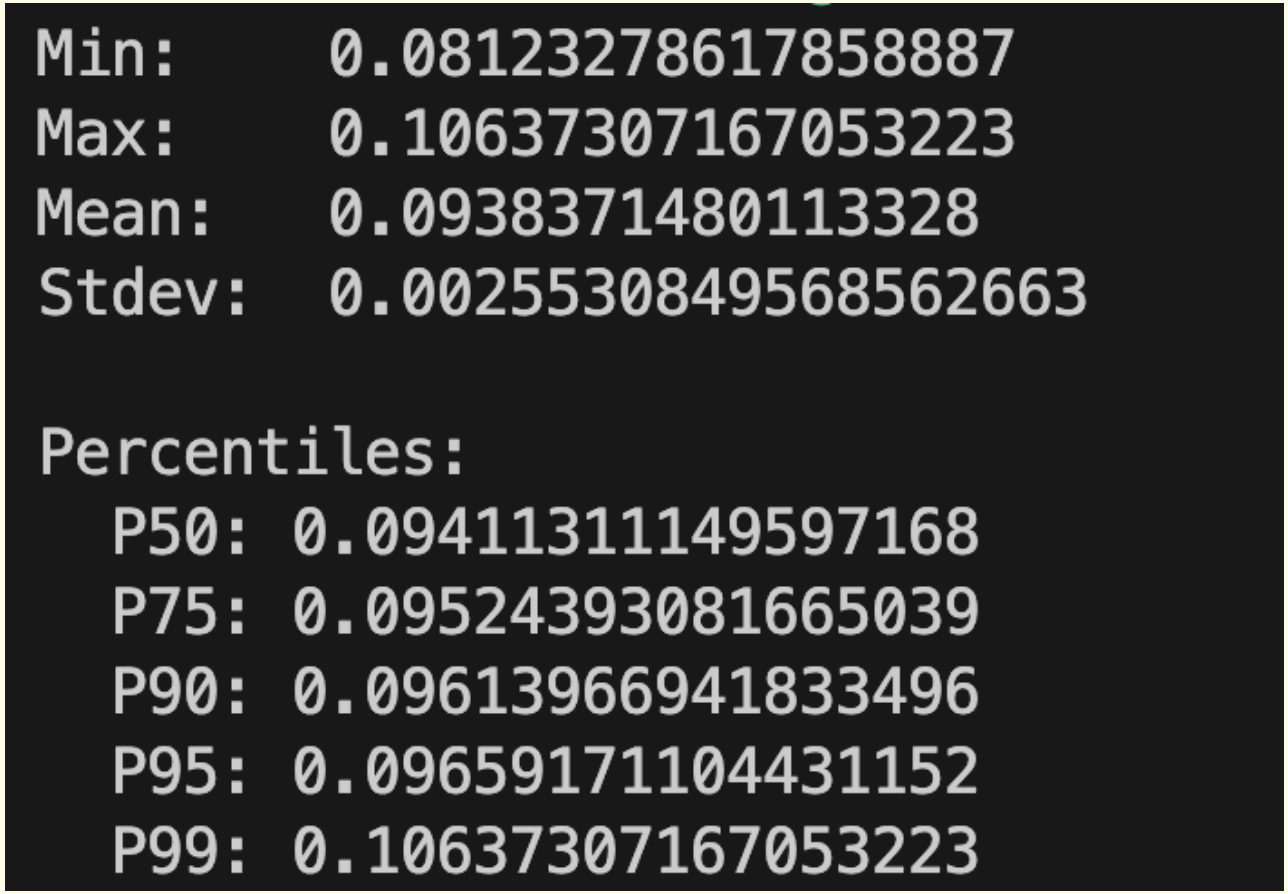


Figure: Decode Stats, 500 requests, single family, 1300 tokens shared context

Batch Decomposition - 500 requests - five families

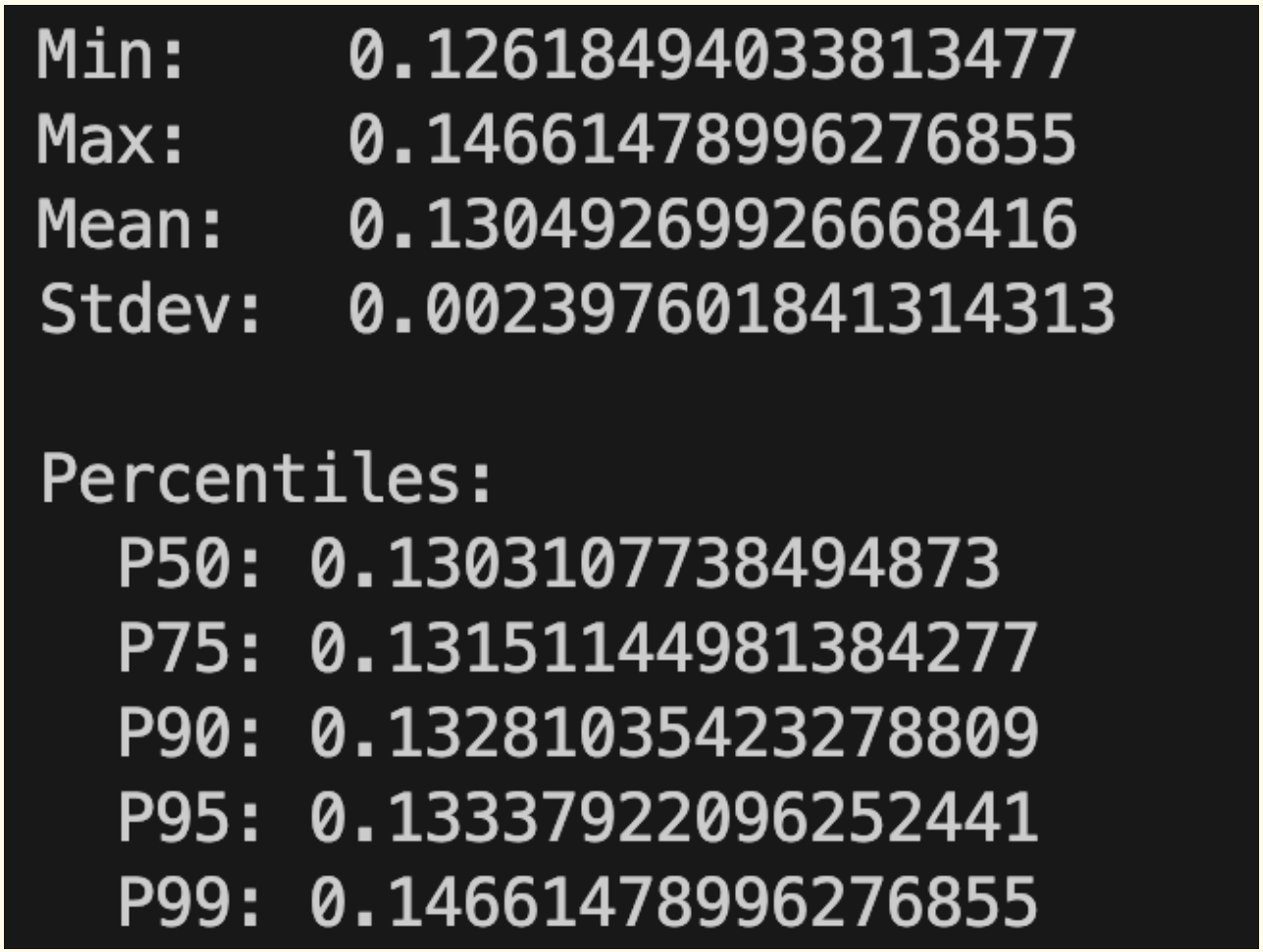


Figure: Decode Stats, 500 requests, five families, 1300 tokens shared context

Key Propositions



Just to summarize, we have the following key propositions:

- We can operate in a dynamic workload - where requests are being added continuously, unlike BatchLLM
- The ChunkedHashTree can work on arbitrarily long requests, with significantly low CPU overhead (can be optimized more)
- We know on when to stop, basically if the prefixes change by a huge amount on adding the next request, we will stop adding stuff at that point, even if we didn't hit `max_batch_size` parameter (missing in all the existing approaches, we know of)
- Our benefits are based on L2 cache purity, instead of being benefitted from Radix Tree Node LRU Eviction from HBM

Simulator Design



- Software model of L2 cache behaviour during batched LLM inference
- Multiple concurrent LLM requests
- Accessing KV cache blocks
- Over many decode iterations
- With/without hardware prefetching

Parameters we can tweak



- Number of families: 1/2/5
- Requests per family: we fix the scheduler batch size to 500, so 500/250/100 requests per family
- Blocks per request: vLLM's block stores 16 tokens, assuming a shared context of 6400 tokens: 400 blocks + 1 block for the distinct suffix (query itelf)
- Decode iterations: 100
- L2 cache size: For our GPU it is 96MB. A shared context of 1300 tokens amounts to 180MB for LLama3.1-8B model. So, 96MB amounts to 44 blocks roughly, out of which we can assume that we have 25 blocks reserved for KV caches (and others for weights/activation tensors etc)
- Number of parallel processing units: We have 142 SMs, and each SM has 64 warps, however each attention kernel might use multiple warps, so let us assume 200 parallel processing units

Simulation Flow



- The simulator runs for a fixed number of decode iterations
- In each iteration:
  1. All requests are reset to initial state
  2. A batch of requests is scheduled to run in parallel
  3. Each scheduled request generates a sequence of memory accesses
  4. Each access is processed by the L2 cache model
  5. Cache hit/miss statistics are updated
  6. Prefetching is triggered
- This loop repeats for 100 decode iterations

L2 Cache Model



- Modeled as a fixed-size LRU cache
- Capacity: 25 KV blocks
- For every block access:
  - If block is in cache $\rightarrow$ demand hit
  - If not in cache $\rightarrow$ demand miss + DRAM access
  - Block is inserted and LRU order updated
- Evicted prefetched blocks are counted as wasted

Stride-Based Prefetching



- Maintains a small history of recent accesses
- Computes stride between consecutive blocks (mostly 1)
- If the same stride repeats with high confidence:
  - Predict future blocks using that stride
  - Issue prefetches ahead of time

Spatial Region Prefetching



- Spatial locality in address space
- When block i is accessed:
  - Also prefetch nearby blocks
  - $i+1, i+2, \ldots$ within a small window
- Useful when:
  - adjacent KV blocks are likely to be reused
  - access patterns are not perfectly sequential

Simulator stats - 1 family



Without Prefetching

Metric	Value
Cache Hit Rate	66.96%
DRAM Accesses	6,625,496
Demand Hits	13,424,504
Demand Misses	6,625,496
Total Accesses	20,050,000

With Prefetching

Metric	Value
Cache Hit Rate	64.25%
DRAM Accesses	22,971,098
Demand Hits	12,881,255
Demand Misses	7,168,745
Total Accesses	20,050,000
Useful Prefetches	4,196,846
Wasted Prefetches	11,605,500
Prefetch Accuracy	26.56%
Stride Prefetches	2,172,430
Region Prefetches	40,000,000
Next-Line Prefetches	0

Simulator Stats - 5 families



Without Prefetching

Metric	Value
Cache Hit Rate	15.08%
DRAM Accesses	17,027,080
Demand Hits	3,022,920
Demand Misses	17,027,080
Total Accesses	20,050,000

With Prefetching

Metric	Value
Cache Hit Rate	14.53%
DRAM Accesses	53,019,202
Demand Hits	2,912,894
Demand Misses	17,137,106
Total Accesses	20,050,000
Useful Prefetches	1,764,442
Wasted Prefetches	34,117,642
Prefetch Accuracy	4.92%
Stride Prefetches	1,876,190
Region Prefetches	40,000,000
Next-Line Prefetches	0

Simulator Stats - without prefetching



1 family

Metric	Value
Cache Hit Rate	66.96%
DRAM Accesses	6,625,496
Demand Hits	13,424,504
Demand Misses	6,625,496
Total Accesses	20,050,000

5 families

Metric	Value
Cache Hit Rate	15.08%
DRAM Accesses	17,027,080
Demand Hits	3,022,920
Demand Misses	17,027,080
Total Accesses	20,050,000

Simulator Stats - with prefetching



1 family

Metric	Value
Cache Hit Rate	64.25%
DRAM Accesses	22,971,098
Demand Hits	12,881,255
Demand Misses	7,168,745
Total Accesses	20,050,000
Useful Prefetches	4,196,846
Wasted Prefetches	11,605,500
Prefetch Accuracy	26.56%
Stride Prefetches	2,172,430
Region Prefetches	40,000,000
Next-Line Prefetches	0

5 families

Metric	Value
Cache Hit Rate	14.53%
DRAM Accesses	53,019,202
Demand Hits	2,912,894
Demand Misses	17,137,106
Total Accesses	20,050,000
Useful Prefetches	1,764,442
Wasted Prefetches	34,117,642
Prefetch Accuracy	4.92%
Stride Prefetches	1,876,190
Region Prefetches	40,000,000
Next-Line Prefetches	0



Thank You